

Dimensionality Reduction & Text Embeddings

Weeks 3-4: From Distributional Semantics to Word Vectors

PSYC 51.07: Models of Language and Communication

December 28, 2025



1.  **Distributional Semantics:** The Foundation
2.  **Dimensionality Reduction:** PCA, UMAP, t-SNE
3.  **Classic Methods:** LSA & LDA
4.  **Modern Topic Modeling:** Top2Vec, BERTopic
5.  **Neural Word Embeddings:** Word2Vec, GloVe, FastText
6.  **Preview:** Transformer Embeddings
7.  **Assignment 3:** Hands-on Practice

Goal: Understand how machines learn to represent meaning

The Fundamental Question 🤔

How do we represent the meaning of words in a way that computers can understand?

How do we represent the meaning of words in a way that computers can understand?

Traditional Approach:

- Dictionaries
- Taxonomies (WordNet)
- Manual feature engineering

Distributional Approach:

- Learn from data
- Context-based
- Scalable

“You shall know a word by the company it keeps”

— J.R. Firth (1957)

"You shall know a word by the company it keeps"

— J.R. Firth (1957)

Key Idea: Words that appear in similar contexts tend to have similar meanings.

Example

- "The **cat** sat on the mat"
- "The **dog** sat on the mat"
- "The **kitten** sat on the mat"

→ *cat, dog, kitten* are semantically related

References: Boleda (2020), Grand et al. (2022)

Cognitive vs. Computational Perspectives

Human Cognition:

- Grounded in experience
- Multimodal (vision, sound, touch)
- Embodied understanding
- Cultural context
- Emotional associations

How do YOU represent "coffee"?

Cognitive vs. Computational Perspectives

Human Cognition:

- Grounded in experience
- Multimodal (vision, sound, touch)
- Embodied understanding
- Cultural context
- Emotional associations

Computational Models:

- Patterns in text data
- Co-occurrence statistics
- Vector representations
- Distributional semantics
- Learned from corpora

How do YOU represent "coffee"?

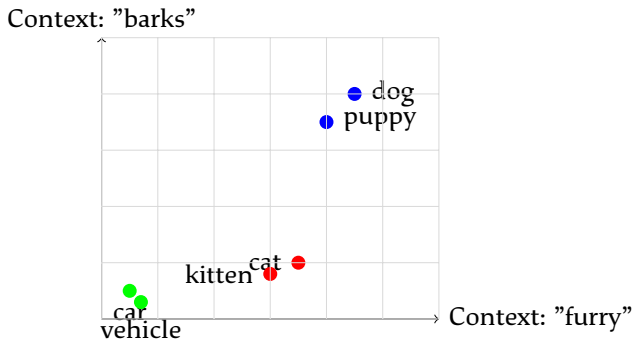
Model represents "coffee" by its linguistic context

Question: What does *semantic similarity* really mean?

Reference: Grand et al. (2022) - "Semantic projection recovers rich human knowledge"

From Words to Vectors

Goal: Represent each word as a point in high-dimensional space



Problem: Real vocabularies have 10,000+ words, contexts are even more numerous!

→ We need **dimensionality reduction** 

Why Dimensionality Reduction?



The Curse of Dimensionality:

- Vocabulary: 50,000 words
- Context features: 50,000+
- Matrix: $50k \times 50k = 2.5B$ entries!
- Sparse (mostly zeros)
- Computationally expensive
- Hard to visualize

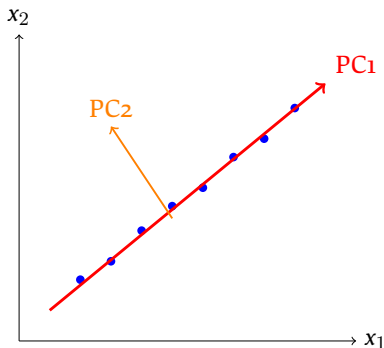
Solution:

- Compress to ~50-300 dimensions
- Preserve semantic relationships
- Dense representations
- Computationally efficient
- Easier to work with
- Can visualize in 2D/3D!

50,000 dimensions $\rightarrow$ 100 dimensions ✨

Principal Component Analysis (PCA)

Intuition: Find the directions of maximum variance



Algorithm:

1. Center the data
2. Compute covariance matrix
3. Find eigenvectors
4. Project onto top k eigenvectors

Properties:

- Linear transformation
- Preserves global structure
- Fast & interpretable

Limitation: Assumes linear relationships (often not true for text!)

UMAP: Uniform Manifold Approximation & Projection

Better for visualizing complex, non-linear structure in text data

Key Advantages:

- Preserves both local & global structure
- Non-linear dimensionality reduction
- Faster than t-SNE
- Theoretically grounded
- Great for visualization

Use Cases:

- Visualizing word embeddings
- Topic modeling
- Clustering analysis
- Exploratory data analysis

```
import umap
from sklearn.datasets import
    load_digits

# Example: Reduce to 2D
reducer = umap.UMAP(
    n_components=2,
    n_neighbors=15,
    min_dist=0.1
)

embedding = reducer.fit_transform(
    word_vectors
)

# Plot
plt.scatter(
    embedding[:, 0],
    embedding[:, 1],
    c=labels
)
```

Reference: McInnes & Healy (2018) - UMAP: Uniform Manifold Approximation and Projection

PCA vs. UMAP vs. t-SNE

Property	PCA	t-SNE	UMAP
Linear/Non-linear	Linear	Non-linear	Non-linear
Speed	Fast ⚡	Slow 🐌	Fast ⚡
Global structure	☐	☐	☐
Local structure	~	☐	☐
Deterministic	☐	☐	~
Scalability	Excellent	Poor	Good

Recommendation

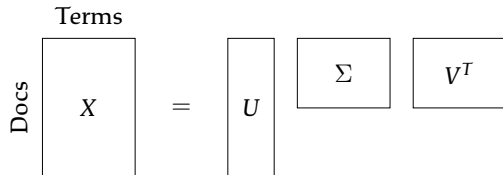
- **PCA:** First pass, quick exploration, preprocessing
- **t-SNE:** 2D/3D visualization (small datasets)
- **UMAP:** Best of both worlds - use for most NLP tasks!

Latent Semantic Analysis (LSA)

The OG of semantic embeddings (1990)

Algorithm:

1. Build term-document matrix X
2. Apply TF-IDF weighting
3. Perform SVD: $X = U\Sigma V^T$
4. Keep top k dimensions
5. Use U_k as word embeddings


$$\begin{matrix} \text{Terms} \\ \boxed{X} \end{matrix} \begin{matrix} \text{Docs} \\ \end{matrix} = \begin{matrix} \boxed{U} \end{matrix} \begin{matrix} \boxed{\Sigma} \end{matrix} \begin{matrix} \boxed{V^T} \end{matrix}$$

Key Innovation:

- Discovers latent topics
- Solves synonymy problem
- Matrix factorization

Limitations:

- Linear relationships only
- Bag-of-words (no order)
- Computationally expensive
- No polysemy handling

Reference: Deerwester et al. (1990) - Indexing by Latent Semantic Analysis

Latent Dirichlet Allocation (LDA)

Probabilistic topic modeling

Generative Story:

1. For each document:
 - Draw topic distribution $\theta \sim \text{Dir}(\alpha)$
2. For each word in document:
 - Choose topic $z \sim \text{Mult}(\theta)$
 - Choose word $w \sim \text{Mult}(\phi_z)$

Key Ideas:

- Documents are mixtures of topics
- Topics are distributions over words
- Bayesian inference

```
from sklearn.decomposition import
    LatentDirichletAllocation

# Fit LDA model
lda = LatentDirichletAllocation(
    n_components=10, # topics
    random_state=42
)

doc_topics = lda.fit_transform(
    doc_term_matrix
)

# Print top words per topic
for idx, topic in enumerate(
    lda.components_
):
    top_words = [
        vocab[i] for i in
        topic.argsort()[-10:]
    ]
    print(f"Topic {idx}: {top_words
        }")
```

Reference: Blei et al. (2003) - Latent Dirichlet Allocation

LDA Example: Topic Discovery

Example Topics from News Corpus:

Topic 1: Politics

government, election, president, vote, policy, senate, congress, party

Topic 2: Sports

game, team, player, score, win, season, coach, championship

Topic 3: Technology

software, computer, data, algorithm, system, internet, code, AI

Topic 4: Finance

market, stock, price, investment, economy, trade, bank, profit

Modern Topic Modeling

Combining neural embeddings with topic models

Top2Vec:

- Embed documents with Doc2Vec
- Cluster in embedding space
- Topics = cluster centroids
- Automatic topic number detection
- No stopword removal needed!

```
from top2vec import Top2Vec

model = Top2Vec(
    documents,
    embedding_model='doc2vec'
)

topics = model.get_topics()
```

BERTopic:

- Embed with transformers (BERT)
- Reduce with UMAP
- Cluster with HDBSCAN
- Topics via TF-IDF on clusters
- State-of-the-art results!

```
from bertopic import BERTopic

model = BERTopic(
    language="english",
    calculate_probabilities=True
)

topics, probs = model.fit_transform(
    documents
)
```

Advantage: Leverage power of pre-trained neural networks!

2013: Everything changed

Key Innovation:

- Neural network approach
- Predict context from word (CBOW)
- Predict word from context (Skip-gram)
- Dense, low-dimensional vectors
- Captures semantic relationships
- Fast training

Famous Result:

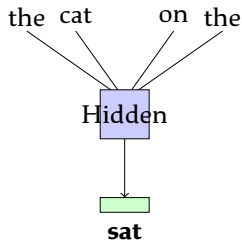
$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

Other Examples:

- $\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Italy}} \approx \vec{\text{Rome}}$
- $\vec{\text{walking}} - \vec{\text{walk}} + \vec{\text{swim}} \approx \vec{\text{swimming}}$

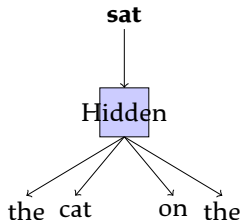
Reference: Mikolov et al. (2013a,b) - *Efficient Estimation of Word Representations*

CBOW (Continuous Bag of Words):



Predict center word from context

Skip-gram:



Predict context from center word

Training Trick: Negative sampling - don't update all weights!

- Sample a few "negative" examples (words that don't appear in context)
- Much faster than full softmax over entire vocabulary

```
from gensim.models import Word2Vec
import gensim.downloader as api

# Load pre-trained model
model = api.load('word2vec-google-news-300')

# Find similar words
similar = model.most_similar('computer', topn=5)
# Output: [('computers', 0.72), ('laptop', 0.69), ...]

# Analogy: king - man + woman = ?
result = model.most_similar(
    positive=['king', 'woman'],
    negative=['man'],
    topn=1
)
# Output: [('queen', 0.71)]
```

Combining count-based & prediction-based methods

Key Insight:

- Word2Vec uses local context windows
- But global statistics matter too!
- Ratios of co-occurrence probabilities are meaningful

Example:

- $P(\text{solid}|\text{ice})$ is high
- $P(\text{solid}|\text{steam})$ is low
- $\frac{P(\text{solid}|\text{ice})}{P(\text{solid}|\text{steam})}$ is very high!

Objective Function:

$$J = \sum_{i,j=1}^V f(X_{ij})(\vec{w}_i^T \tilde{\vec{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

where:

- X_{ij} = co-occurrence count
- f = weighting function
- $\vec{w}_i$ = word vector
- $\tilde{\vec{w}}_j$ = context vector

Result:

- Similar performance to Word2Vec
- More interpretable
- Faster training on large corpora

Reference: Pennington et al. (2014) - GloVe: Global Vectors for Word Representation

FastText: Subword Information

Word2Vec's limitation: Can't handle unknown words!

Innovation:

- Represent words as bag of character n-grams
- Word = sum of n-gram vectors
- Can generate embeddings for OOV words
- Captures morphology

Example:

- Word: "where"
- 3-grams: <wh, whe, her, ere, re>
- $where = \sum n\text{-gram}$

```
from gensim.models import FastText

# Train FastText model
model = FastText(
    sentences,
    vector_size=100,
    window=5,
    min_count=1,
    min_n=3, # min n-gram length
    max_n=6  # max n-gram length
)

# Works for unknown words!
vec = model.wv['unknownword']

# Great for morphologically rich
# languages
# e.g., German, Finnish, Turkish
```

Advantages:

- Handles rare words better
- Morphological awareness
- More robust

Reference: Bojanowski et al. (2017) - Enriching Word Vectors with Subword Information

Embedding Methods Comparison

Method	Year	Type	OOV	Context	Speed	Dims
LSA	1990	Count	□	Static	Medium	100-300
LDA	2003	Probabilistic	□	Static	Slow	10-100
Word2Vec	2013	Neural	□	Static	Fast	100-300
GloVe	2014	Hybrid	□	Static	Fast	50-300
FastText	2017	Neural	□	Static	Fast	100-300
BERT	2018	Transformer	□	Contextual	Slow	768+

Key Takeaway

- **Static embeddings:** One vector per word (Word2Vec, GloVe, FastText)
- **Contextual embeddings:** Different vectors based on context (BERT, GPT)
- We'll cover contextual embeddings in detail next week!

Visualizing Embeddings with UMAP

Example: Word2Vec + UMAP

```
import umap
import matplotlib.pyplot as plt

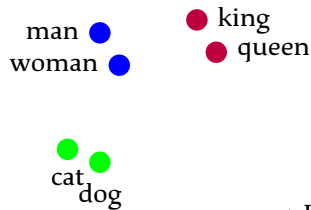
# Get word vectors
words = ['king', 'queen', 'man',
        'woman', 'cat', 'dog']
vectors = [model[w] for w in words]

# Reduce to 2D
reducer = umap.UMAP(n_components=2)
embedding_2d = reducer.fit_transform(
    vectors)
```

What you'll see:

- Semantic clusters
- Gender relationships
- Animal groupings
- Analogical structures

UMAP-2



UMAP-1

How Do We Evaluate Embeddings?

Intrinsic Evaluation:

- **Word Similarity:** Compare to human judgments
 - WordSim-353
 - SimLex-999
- **Analogies:** $a : b :: c : d$
 - Google Analogy Dataset
 - Semantic & Syntactic
- **Clustering:** Do similar words cluster?

Extrinsic Evaluation:

- Use embeddings in downstream tasks
- **Classification:** Sentiment analysis
- **NER:** Named entity recognition
- **QA:** Question answering
- **MT:** Machine translation

Better embeddings → Better task performance

Important Note

No single "best" embedding for all tasks! Choice depends on:

- Domain (general vs. specialized)
- Task requirements
- Computational constraints

Limitations of Static Embeddings

The fundamental problem: One vector per word

Example: "bank"

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)

Both get the **same vector**! This is wrong.

Other Issues:

- No compositional semantics
- Can't handle phrases well
- Bias in embeddings
- Limited to training corpus
- No common sense reasoning

Solution:

- **Contextual embeddings!**
- Different vector for each occurrence
- Transformer models (BERT, GPT)
- Coming in Week 5-6...

Preview: Transformer Embeddings

The next evolution: Context-aware representations

Static (Word2Vec, GloVe):

```
# Same vector every time
vec1 = model['bank']
vec2 = model['bank']
# vec1 == vec2 (always!)
```

Problem: "bank" in different contexts gets same representation

Contextual (BERT):

```
from transformers import
    AutoModel, AutoTokenizer

model = AutoModel.
    from_pretrained('bert-base-
uncased')
tokenizer = AutoTokenizer.
    from_pretrained('bert-base-
uncased')

sent1 = "river bank"
sent2 = "money bank"
```

Discussion: How Do Humans Represent Meaning? 🤔

Think about the word "coffee"

What comes to mind?

- Visual: brown liquid, mug, beans
- Olfactory: aroma, smell
- Gustatory: taste, bitter
- Tactile: hot, warm cup
- Contextual: morning, caffeine, energy
- Emotional: comfort, alertness
- Cultural: cafe, social

Key Question

How does this compare to how Word2Vec represents "coffee"?

- Word2Vec: "coffee" = [0.23, -0.45, 0.67, ..., 0.12] (300 numbers)
- Learned purely from text co-occurrence
- Missing grounding in sensory experience!

This is the "symbol grounding problem" in AI

Practical Tips for Using Embeddings

1. Start with pre-trained models

- Word2Vec: Google News (3B words)
- GloVe: Common Crawl (840B tokens)
- FastText: Available in 157 languages

2. Fine-tune if you have domain-specific data

- Medical texts, legal documents, etc.
- Can significantly improve performance

3. Choose dimensions wisely

- More dims = more expressiveness, more data needed
- Typical: 100-300 dimensions

4. Normalize vectors

- Use cosine similarity, not Euclidean distance
- $\text{similarity}(u, v) = \frac{u \cdot v}{||u|| \cdot ||v||}$

5. Be aware of biases

- Embeddings reflect biases in training data
- Gender, race, cultural biases can be encoded

Assignment 3: Exploring Text Embeddings

Hands-on practice with everything we've learned!

Tasks:

1. **Build a term-document matrix** from a corpus
 - Implement TF-IDF weighting
 - Visualize with PCA
2. **Train your own Word2Vec model**
 - Compare CBOW vs Skip-gram
 - Explore word analogies
3. **Visualize embeddings with UMAP**
 - Create 2D plots
 - Identify semantic clusters
4. **Topic modeling with LDA**
 - Extract topics from documents
 - Interpret topic coherence
5. **Compare embedding methods**
 - Word2Vec vs GloVe vs FastText
 - Evaluate on similarity tasks

Due: Check course website for deadline

Key Papers:

- Deerwester et al. (1990) - Latent Semantic Analysis
- Blei et al. (2003) - Latent Dirichlet Allocation
- Mikolov et al. (2013a,b) - Word2Vec
- Pennington et al. (2014) - GloVe
- Bojanowski et al. (2017) - FastText
- McInnes & Healy (2018) - UMAP
- Boleda (2020) - Distributional Semantics and Linguistic Theory
- Grand et al. (2022) - Semantic Projection

Tutorials & Tools:

- HuggingFace Course: Chapters 2.2, 5.6
- Gensim documentation: <https://radimrehurek.com/gensim/>
- UMAP documentation: <https://umap-learn.readthedocs.io/>
- Word2Vec tutorial: <https://rare-technologies.com/word2vec-tutorial/>

What we learned:

1. **Distributional Hypothesis:** Words in similar contexts have similar meanings
2. **Dimensionality Reduction:** PCA (linear), UMAP (non-linear), t-SNE (visualization)
3. **Classic Methods:**
 - LSA: SVD on term-document matrix
 - LDA: Probabilistic topic modeling
4. **Neural Embeddings:**
 - Word2Vec: CBOW & Skip-gram
 - GloVe: Global co-occurrence statistics
 - FastText: Subword information
5. **Limitations:** Static embeddings can't handle polysemy
6. **Next:** Contextual embeddings (BERT, transformers)

From words to vectors, from vectors to meaning! 

Next Week: Neural Language Models

Building towards modern LLMs

- Recurrent Neural Networks (RNNs)
- Long Short-Term Memory (LSTMs)
- Sequence-to-Sequence models
- Attention mechanisms
- The path to transformers...

Questions? 